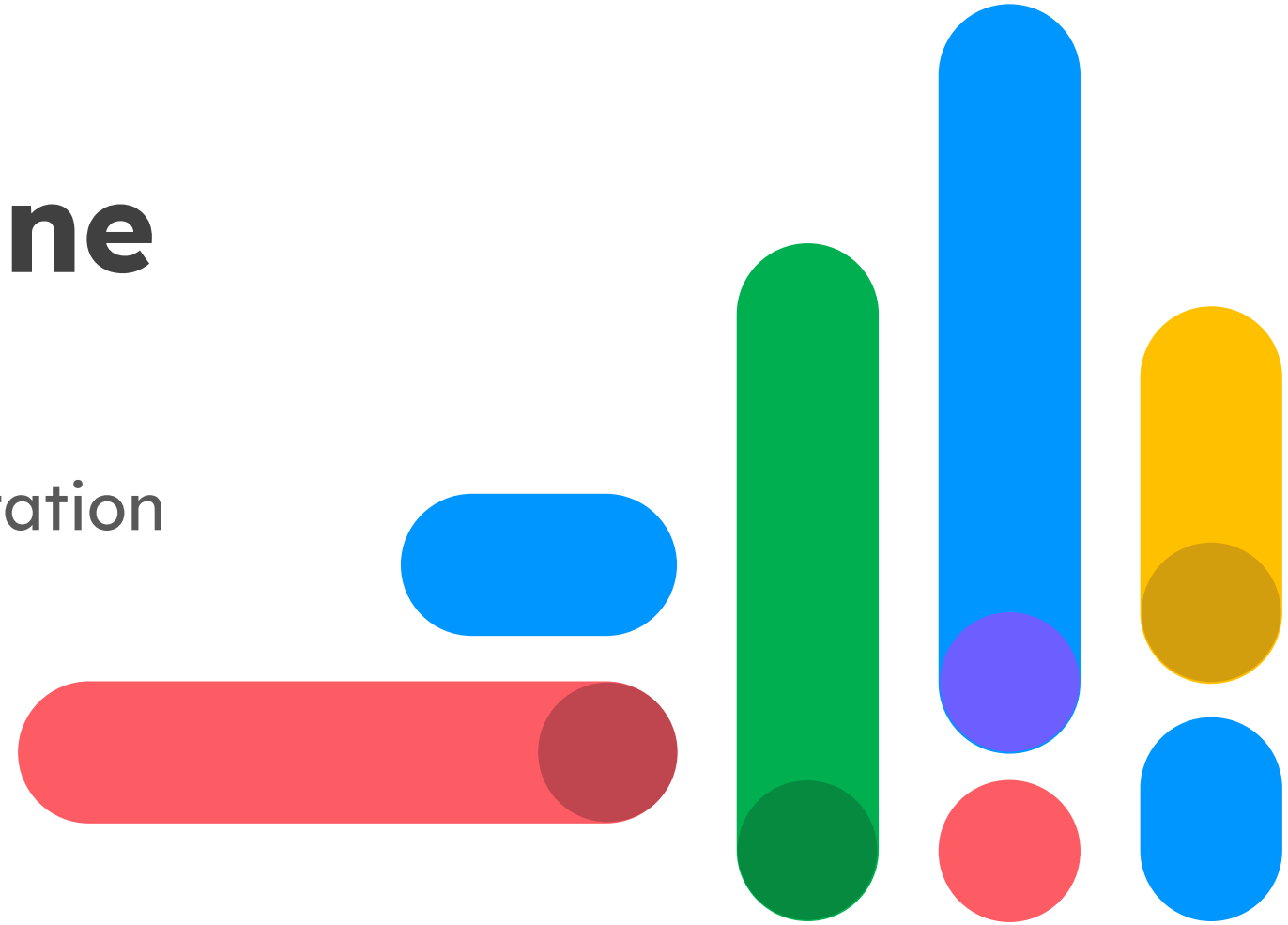
Deep Learning

Applied Machine Learning

Computer Visions :: Segmentation

Assistant Professor | **Chun-Hsiang Chan**

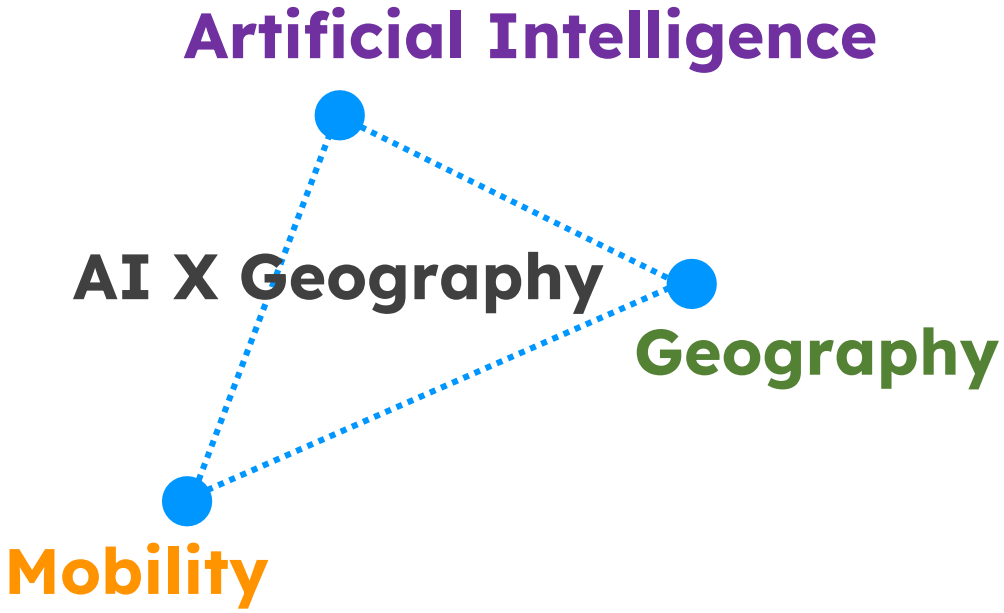
Department of Geography
National Taiwan Normal University



Outline

#GeoAI #Mobility #Geography #STDS

01 Semantic Segmentation
02 Encoder-Decoder
03 Skip Connections
04 U-Net
05 Instance Segmentation
06 Mask R-CNN
07 Evaluation Metrics
08 References



Semantic Segmentation

Semantic Segmentation
Semantic segmentation predicts a category for every pixel.
For an input:

$$I \in \mathbb{R}^{H \times W \times 3}$$

with K mutually exclusive categories, a model can produces

$$Z \in \mathbb{R}^{H \times W \times K}$$

After Softmax

$$p_{i,k} = \frac{\exp(z_{i,k})}{\sum_{j=1}^{K} \exp(z_{i,j})} \Rightarrow \hat{y}_i = arg\max_k p_{i,k}$$

The model may first produce lower-resolution predictions internally and then upsample them to the required output size.

Semantic Segmentation

How does this differ from ordinary classification?

Ordinary classification may aggregate the features of an entire image into one vector and output a category.

Segmentation must preserve the relationship between locations and predictions because spatial correspondence is part of the task.

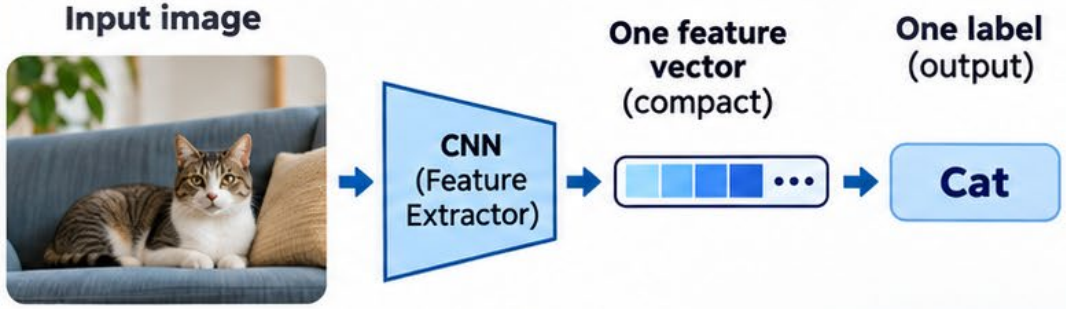
The color of a single pixel is insufficient to determine its category. The model also needs nearby shapes and broader context.

Semantic Segmentation

How does this differ from ordinary classification?

Ordinary Classification

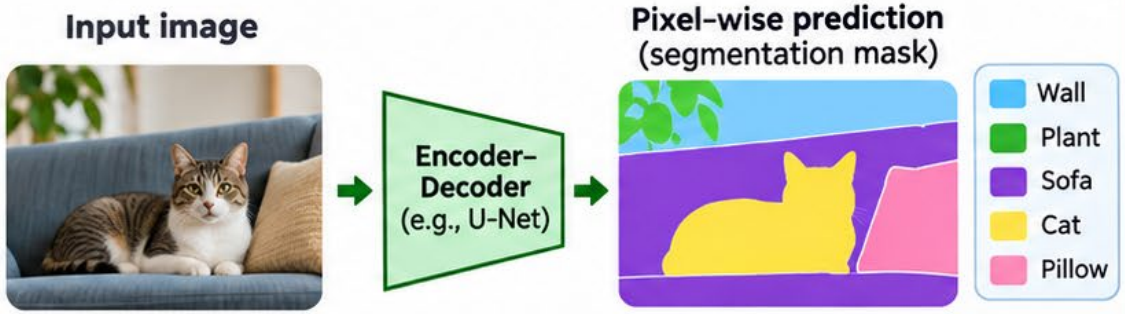
The model answers: **"What is in the image?"**



- ✓ Whole image is **aggregated** into one representation.
- ✓ Spatial details are mostly **compressed away**.

Semantic Segmentation

The model answers: **"What class is each pixel?"**



- ✓ Prediction must **preserve where** each class is located.
- ✓ Uses **nearby shapes** and **broader context** to decide each pixel's class.

Semantic Segmentation

Semantic Segmentation

Setting	Example	Common output design
Binary segmentation	Defect/background	One sigmoid channel
Mutually exclusive multiclass segmentation	Road/person/car/sky	Multichannel softmax
Multi-label segmentation	A pixel can satisfy multiple labels	Multiple independent sigmoids

Semantic segmentation identifies which pixels belong to “person,” but it does not directly determine how many individual people those pixels represent.

It is useful for analyzing class regions and areas.

Counting or measuring individual objects generally requires instance-level information.

Encoder-Decoder

Encoder–decoder is a general architectural pattern. U-Net is one specific implementation, which we will introduce later.

It can be expressed as:

$$z = E(I); \hat{Y} = D(z)$$

where E is the encoder, z is an intermediate representation, D is the decoder, and $\hat{Y}$ is the output.

Encoder-Decoder

What does the encoder do?

The encoder converts the input into a representation useful for the task.

In a typical CNN segmentation network:

1) **Spatial dimensions** progressively **decrease**.
2) **Channel counts** progressively **increase**.
3) **Features** integrate information **over larger regions**.

Encoder-Decoder

What does the decoder do?

The decoder transforms the intermediate representation into the required output structure.

For segmentation, it often progressively increases spatial resolution to produce pixel-level predictions.

A decoder is not an exact mathematical inverse of the encoder. Downsampling may have discarded information. The decoder must use learned relationships and additional features to construct a suitable output.

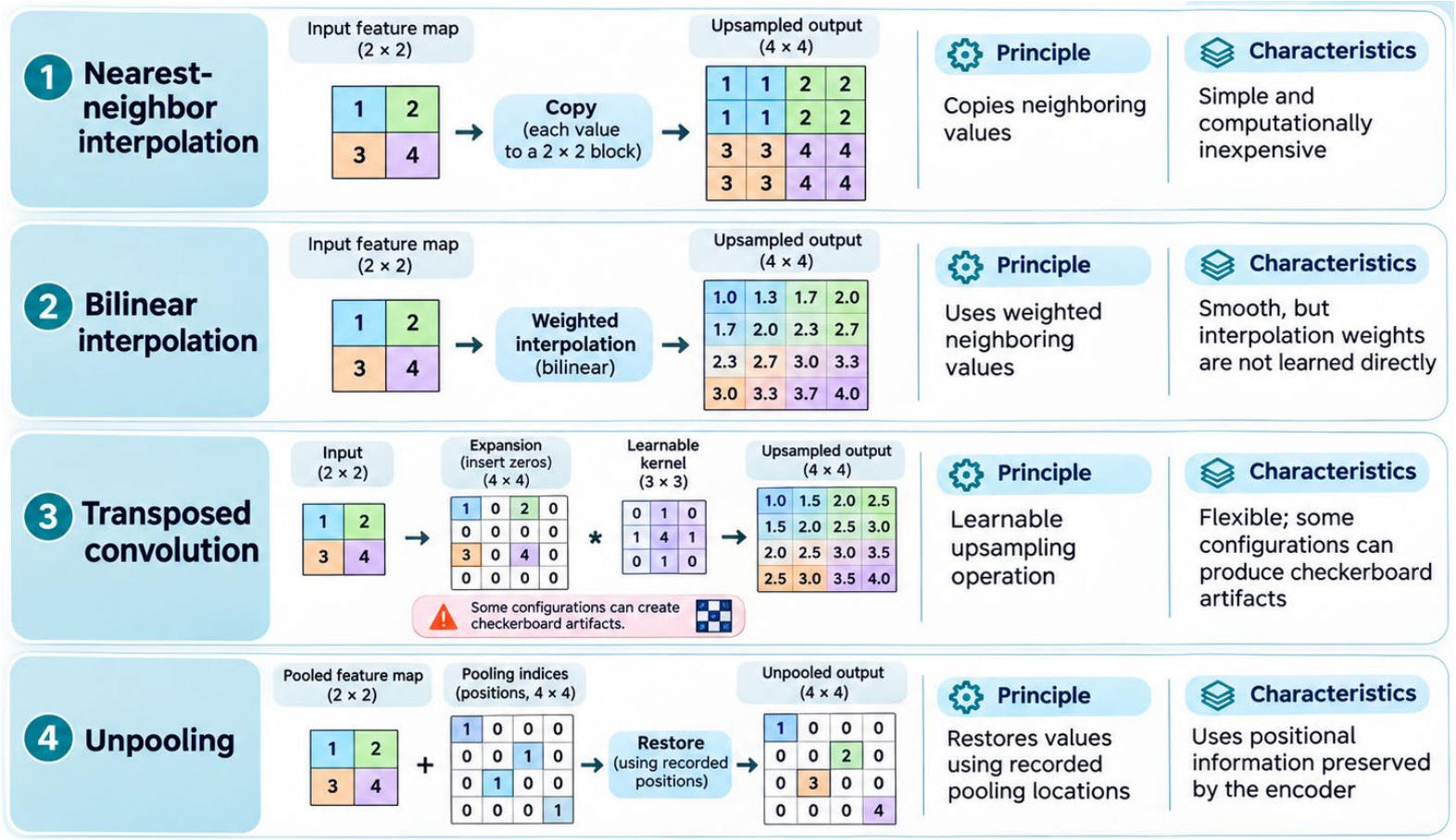
Encoder-Decoder

Common Upsampling Methods

Method	Principle	Characteristics
Nearest-neighbor interpolation	Copies neighboring values	Simple and computationally inexpensive
Bilinear interpolation	Uses weighted neighboring values	Smooth, but interpolation weights are not learned directly
Transposed convolution	Learnable upsampling operation	Flexible; some configurations can produce checkerboard artifacts
Unpooling	Restores values using recorded pooling locations	Uses positional information preserved by the encoder

Encoder-Decoder

Common Upsampling Methods



Skip Connections

A **skip connection** transfers information across some intermediate layers directly to a later part of the network.

It is a connection pattern, not one uniquely defined operation.

Two common purposes are:

- Improving optimization of deep networks.
- Combining features from different levels or scales.

Skip Connections

ResNet :: additive residual connections

A typical ResNet operation is:

$$y = F(x) + x$$

where:

x is the input.

$F(x)$ is the transformation learned by intermediate layers.

y is the result after addition.

Skip Connections

ResNet :: Additive Residual Connections

If the dimensions differ, a projection can be used:

$$y = F(x) + W_s x$$

The model can learn the correction that should be added to the original representation.

For a simplified residual unit:

$$\frac{\partial y}{\partial x} = \frac{\partial F(x)}{\partial x} + I$$

The identity path provides a direct route for information and gradients, helping train deep networks. It does not guarantee that all gradient problems disappear, but it improves optimization conditions.

Skip Connections

U-Net: Concatenative Feature Connections

U-Net commonly uses:

$$y = Concat(x_{encode}, x_{decode})$$

If both feature maps have dimensions:

$$64 \times 64 \times 128$$

concatenation along the channel dimension produces:

$$64 \times 64 \times 256$$

Subsequent convolutions learn how to combine the features.

This operation preserves both feature sets and allows later layers to determine how to use them.

Skip Connections

Addition versus concatenation

Aspect	Addition	Concatenation
Operation	Adds corresponding elements	Joins features along the channel dimension
Dimension requirement	Dimensions being added must match	Spatial dimensions must match; channel counts may differ
Channel count	Usually unchanged	Channel counts are added together
Typical purpose	Residual learning	Multilevel feature fusion
Common example	ResNet	U-Net

Skip Connections

Why do skip connections help segmentation?

Deep features may indicate that “a car is here” while retaining only a coarse location.

Shallow features may preserve edges and textures without being sufficient to identify the object.

Combining them allows the model to use semantic information and precise spatial cues together.

Skip Connections

Why do skip connections help segmentation?

However, skip connections have costs:

- Intermediate features must be retained, increasing memory use.
- Concatenation can increase subsequent computation.
- Shallow-layer noise can reach the decoder.
- Features from different levels may require additional transformations before they combine effectively.

Skip connections are often useful, but adding more does not automatically improve performance.

U-Net

U-Net is a segmentation network with an encoder–decoder structure and cross-layer connections. The original U-Net was proposed by Ronneberger, Fischer, and Brox for biomedical image segmentation.

Its name comes from its U-shaped architecture:

- The **left side** progressively **reduces** spatial resolution.
- The **bottom** combines **higher-level** features.
- The **right side** progressively **restores** output resolution.

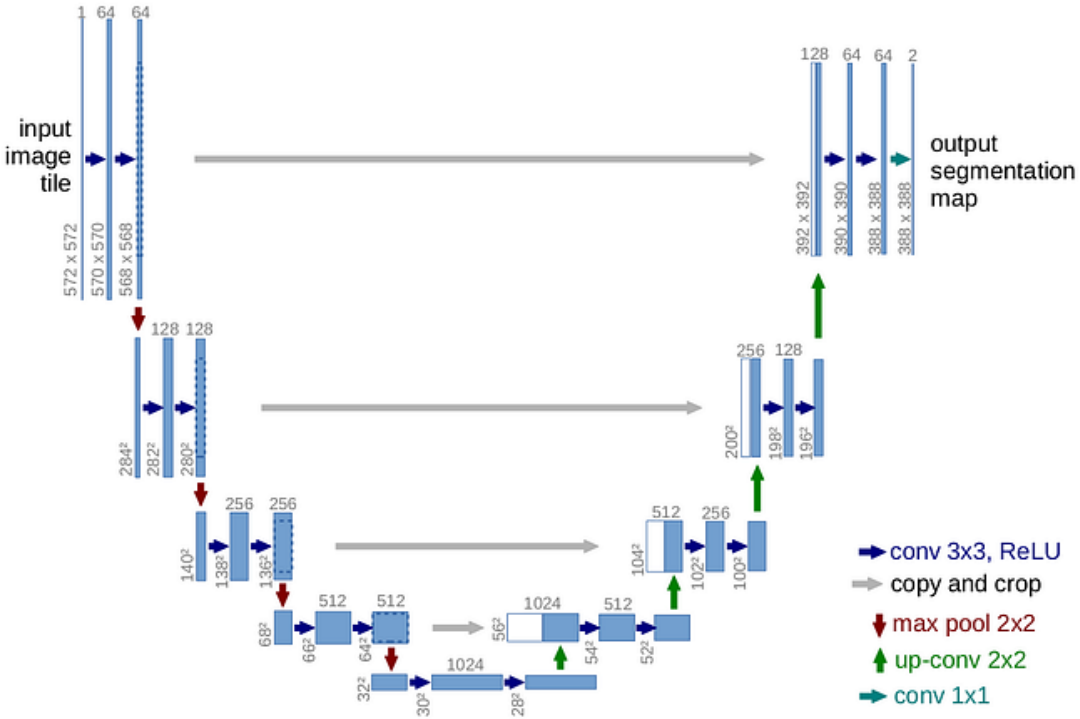
Connections between corresponding scales transfer features across the network.

U-Net

How does U-Net work?

The encoder uses convolutions and downsampling to integrate information over increasingly large regions.

The decoder upsamples lower-resolution features and combines them with details from the encoder to gradually produce a refined segmentation.



U-Net

U-Net supplies higher-resolution encoder features directly to the decoder, giving it access to:

- High-level semantic information.
- Low-level spatial details.

A typical fusion operation is:

$$D_l = f_l(Concat[E_l, Up(D_{l+1})])$$

where E_l is an encoder feature map and D_{l+1} is a decoder feature map.

U-Net

The Original U-Net vs Common Implementations

The original U-Net uses unpadded convolutions, so spatial dimensions shrink and skip features must be cropped for alignment. Its architecture diagram shows a (572×572) input and a (388×388) output.

Many later implementations use padding to make matching input and output sizes easier.

The original paper also uses weighted pixel-wise cross-entropy and increases the weight of boundaries between adjacent cells.

U-Net

U-Net & Dice Loss

In later practice, U-Net-like models are often trained with Dice loss or a combination such as:

$$L = \alpha L_{CE} + \beta L_{Dice}$$

One common soft Dice loss is:

$$L_{Dice} = 1 - \frac{2\sum_i p_i y_i + \epsilon}{\sum_i p_i + \sum_i y_i + \epsilon}$$

Here, p_i is the predicted foreground probability and y_i is the foreground label.

U-Net

Practical limitations

1) Storing high-resolution skip features consumes memory.
2) Overfitting remains possible with small datasets.
3) Results depend on data quality, annotation consistency, and training strategy.
4) A standard semantic segmentation output does not automatically provide instance IDs.
5) Therefore, “U-Net is suitable for limited data” does not mean that any small dataset will be sufficient for successful training.

Instance Segmentation

Instance segmentation identifies:

1) The category of each object.
2) The pixels belonging to each object.
3) Different individual objects within the same category.

A typical output is:

$$S = \{(c_i, s_i, M_i)\}_{i=1}^{N}$$

where:

c_i is the category.

s_i is the score.

M_i is the mask for object i.

Instance Segmentation

Suppose three cats are touching one another.

Semantic segmentation may produce one continuous region labeled “cat.”

Although the category is the same, the pixels must be assigned to different objects.

Datasets such as COCO support this task with per-instance mask annotations.



Instance Segmentation

Why not simply apply connected-component analysis to a semantic mask?

Problems arise when:

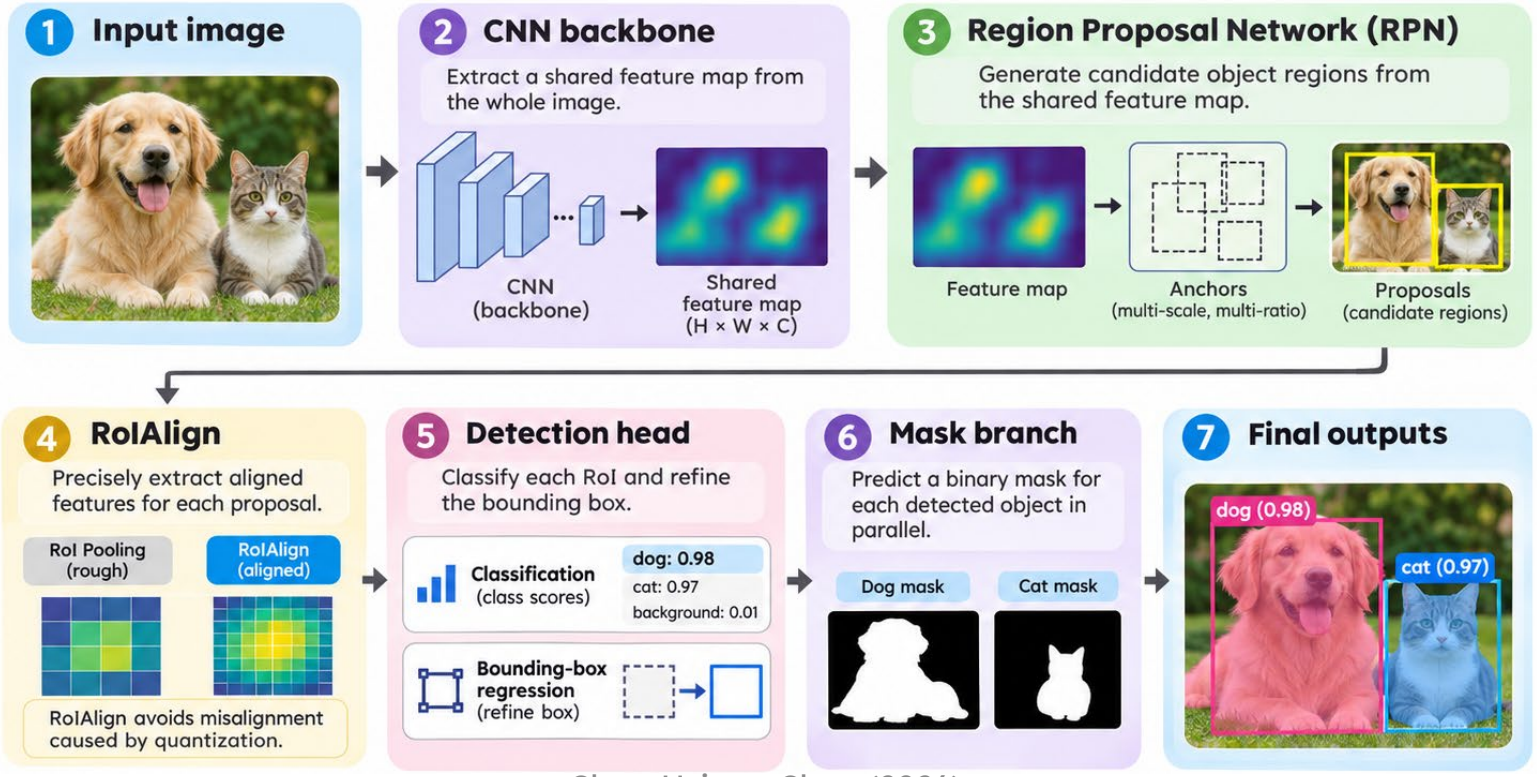
- Two touching objects are treated as one component.
- One occluded object appears as two components.
- Boundary errors merge objects or split one object apart.

A connected component is therefore not necessarily an object instance.

Mask R-CNN

Mask R-CNN can be understood as:

$$Faster\ R-CNN + Mask\ branch + RoI\ Align$$

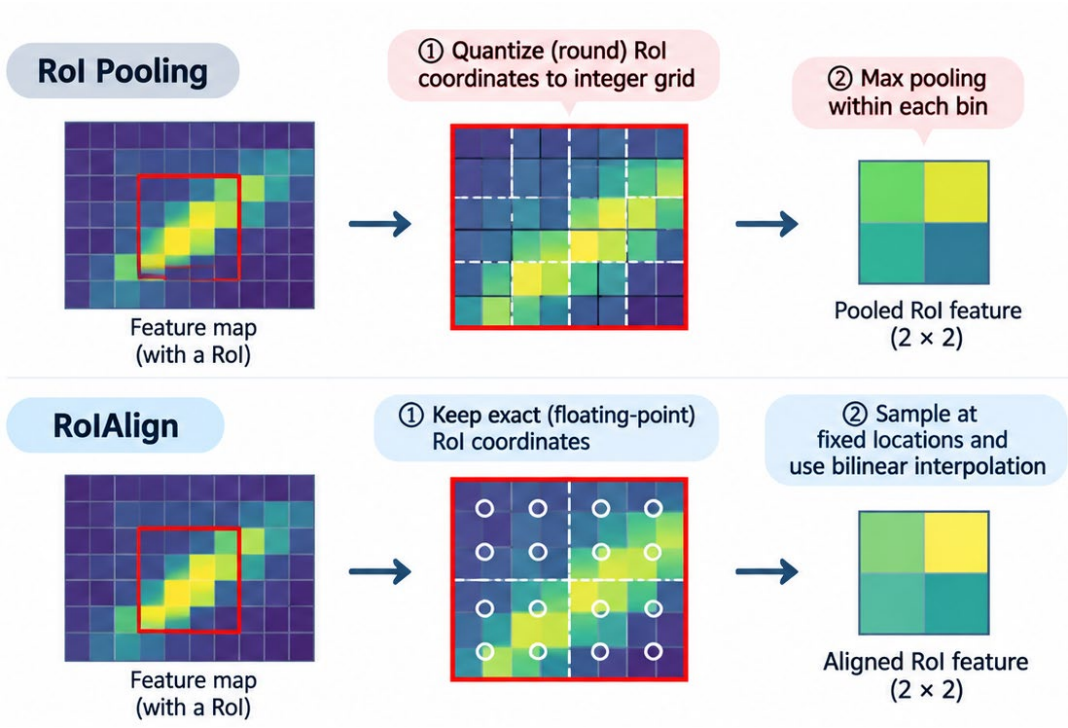


Mask R-CNN

Why is RoI Pooling not ideal?

RoI Pooling quantizes coordinates when mapping regions and dividing them into bins, for example by rounding floating-point positions to integers.

Classification may still recognize “this is a dog,” but pixel-level segmentation requires accurate boundary alignment, making these errors more consequential.



Mask R-CNN

What does RoIAlign do?

RoIAlign avoids this integer quantization of RoI boundaries and sampling positions. It uses bilinear interpolation to obtain features at continuous coordinates. Conceptually, when a sample falls between four feature points, its value is estimated using distance-dependent weights:

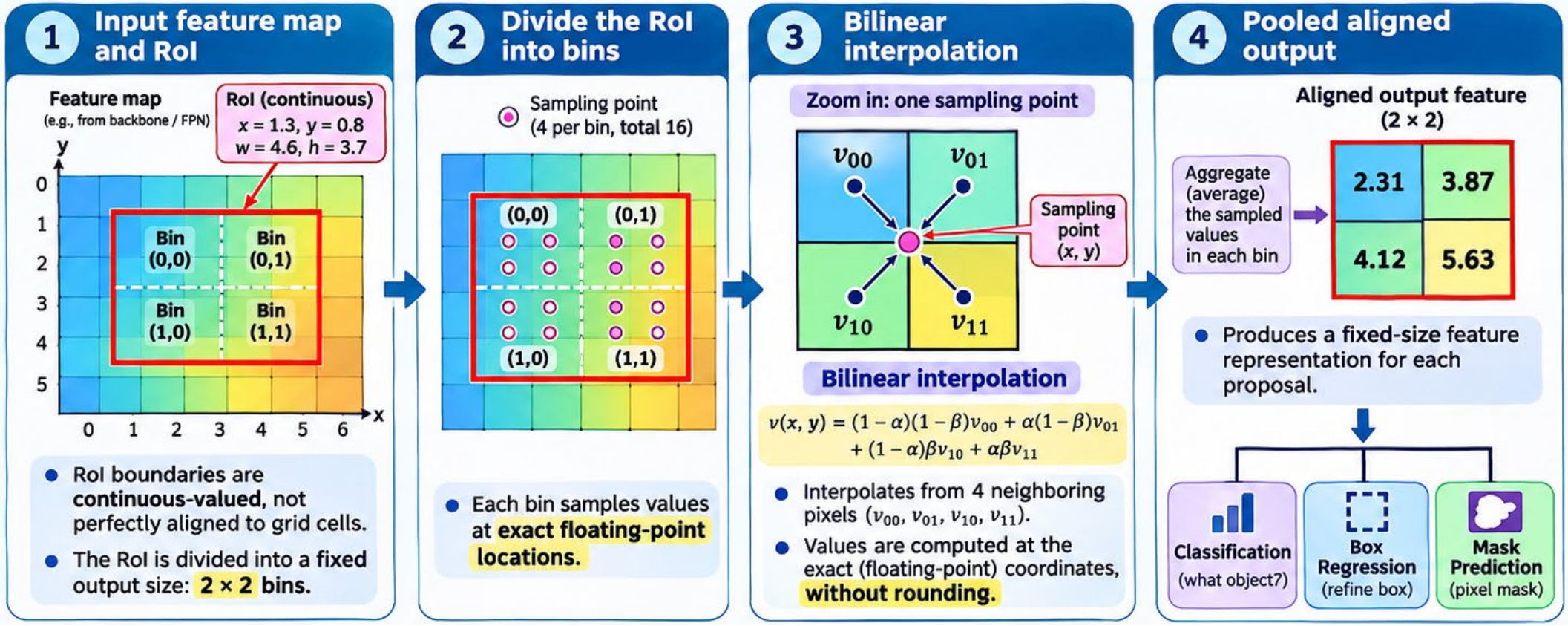
$$f(x, y) = \sum_{i,j} w_{ij}(x, y) f(i, j)$$

The sum involves the neighboring grid points.

RoIAlign does not create image information that was never present. It reduces additional misalignment introduced during feature sampling.

Mask R-CNN

What does RoIAlign do?



Mask R-CNN

What does the mask branch output?

The original design can output $K \times m \times m$ for each RoI, where:

K is the number of object categories.

$m \times m$ is the local mask resolution for each category.

For example, $m = 28$ produces a 28×28 local mask prediction for each category.

During training, a positive RoI's mask loss is computed only for the channel corresponding to its ground-truth category.

During inference, the mask for the predicted category is selected, resized, and placed at the corresponding image location.

Mask R-CNN

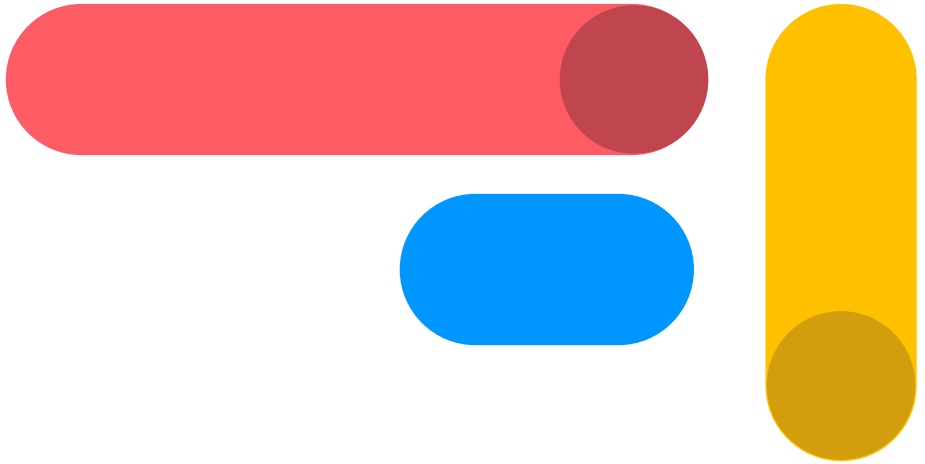
Loss function

The second-stage multitask loss for each RoI can be written as:

$$L = L_{cls} + L_{box} + L_{mask}$$

Training the complete system also includes the RPN losses.

Mask R-CNN



Advantages and limitations

The architecture produces separate masks for individual objects while sharing much of the image feature computation.

Its limitations include:

1) Proposal generation and RoI processing add computation.
2) Objects missed upstream generally cannot be recovered by the mask branch.
3) Fixed-resolution local masks may struggle with extremely fine structures.
4) Small objects provide limited features, making their boundaries harder to recover.

Evaluation Metrics

Classification Metrics :: accuracy, precision, recall, and F1

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precison = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Evaluation Metrics

Detection :: What counts as a true positive?

In a typical setting, a predicted detection must satisfy at least three conditions to count as a TP:

1) Its predicted category is correct.
2) Its IoU with a ground-truth object meets the evaluation threshold.
3) That ground-truth object has not already been matched to another higher-priority prediction.

Suppose one real dog produces three predicted dog boxes. Even if all three boxes are close to the dog, generally only one can count as a TP. The duplicates count as FPs.

Evaluation Metrics

Dice

The Dice coefficient is a measure of the similarity between two sets, A and B. The coefficient ranges from 0 to 1, where 1 indicates that the two sets are identical, and 0 indicates that the two sets have no overlap.

$$Dice = \frac{2|A \cap B|}{|A| + |B|} = \frac{2TP}{2TP + FP + FN} = \frac{2IoU}{1 + IoU}$$

$$IoU = \frac{Dice}{2 - Dice}$$

Mean Dice cannot generally be substituted into the formula to obtain mean IoU, because nonlinear transformations and averaging do not commute.

References

1) R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 580–587.
2) R. Girshick, “Fast R-CNN,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2015.
3) S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
4) J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
5) W. Liu *et al.*, “SSD: Single shot multibox detector,” in *Proc. European Conf. Computer Vision (ECCV)*, 2016, pp. 21–37.
6) T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.

References

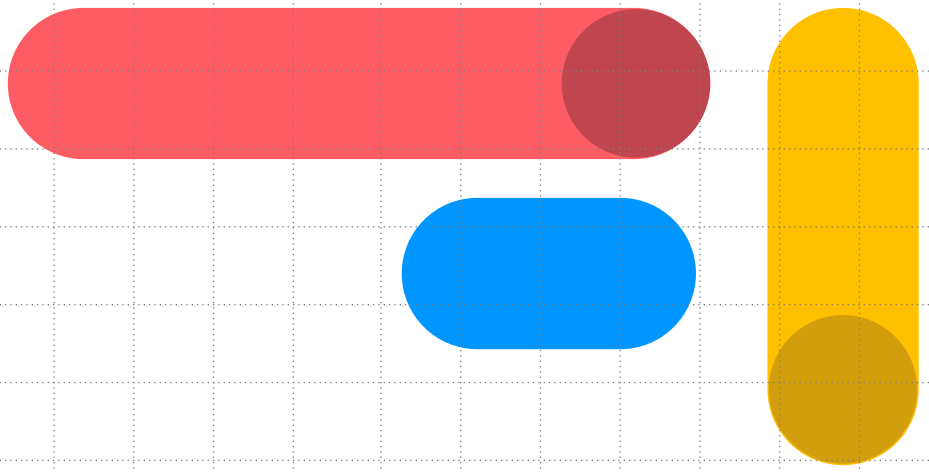
7) H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, “Generalized intersection over union: A metric and a loss for bounding box regression,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019.
8) N. Bodla, B. Singh, R. Chellappa, and L. S. Davis, “Soft-NMS—Improving object detection with one line of code,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017.
9) J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3431–3440.
10) O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
11) V. Badrinarayanan, A. Kendall, and R. Cipolla, “SegNet: A deep convolutional encoder-decoder architecture for image segmentation,” *arXiv preprint arXiv:1511.00561*, 2015.

References

12) K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
13) K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2961–2969.
14) J. Redmon and A. Farhadi, “YOLO9000: Better, faster, stronger,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017.
15) J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” *arXiv preprint arXiv:1804.02767*, 2018.
16) A. Wang *et al.*, “YOLOv10: Real-time end-to-end object detection,” *arXiv preprint arXiv:2405.14458*, 2024.
17) F. Milletari, N. Navab, and S.-A. Ahmadi, “V-Net: Fully convolutional neural networks for volumetric medical image segmentation,” in *Proc. Int. Conf. 3D Vision (3DV)*, 2016.
18) PASCAL VOC, “The PASCAL Visual Object Classes Challenge 2012 (VOC2012) development kit,” online documentation. Accessed: Oct. 21, 2026.
19) COCO Consortium, “COCO API: PythonAPI/pycocotools/cocoeval.py,” GitHub repository, source code. Accessed: Oct. 21, 2026.

References

20) PyTorch Contributors, “torchvision.ops.nms,” *Torchvision documentation*. Accessed: Oct. 21, 2026.
21) scikit-learn Developers, “Metrics and scoring: Quantifying the quality of predictions,” *scikit-learn documentation*. Accessed: Oct. 21, 2026.
22) T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017.



The End

Thank you for your attention!

Email: chchan@ntnu.edu.tw

Website: https://toodou.github.io/